Replicating Marin Datakit: a code-level data-processing manual

Artifact contracts, exact transformations, dependencies, failure checks and an implementable pilot

For the **specific PR → pinned code → data artifact → result** chain, use the artifact-lineage companion. It adds historical production checks, a public HF mixture-search replay and explicit access gaps. See also Marin evaluation and mid/post-training experiments.

Choose a coherent reproduction target

There are three different deliverables:

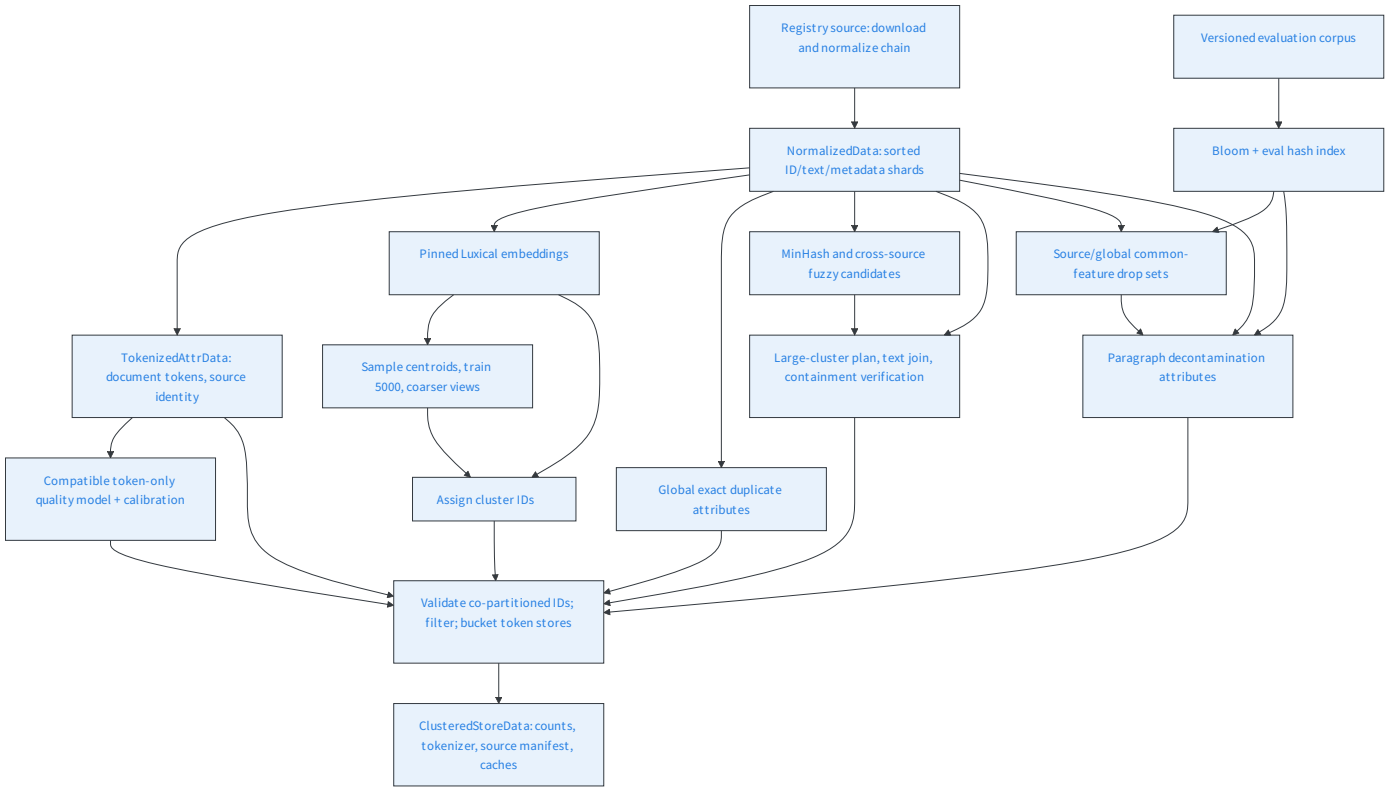
1. **Historical reproduction:** rebuild the corpus/buckets used by a specified Marin experiment, including its source snapshots, tokenizer, embedding/scoring checkpoints, centroids and mixture schedule.
2. **Current-code reproduction:** execute one pinned Datakit DAG with its complete compatible inputs. It can differ materially from the historical report.
3. **Our extension:** reuse the pipeline principles, substitute an accessible corpus/trainer, and add cyber data. This needs new empirical validation; it is not a byte-identical Marin reproduction.

For this walkthrough, code means commit `1d274b52c636b6f2a9177b1d4c14b2f74795a8a6`. The evidence bundle saves inspected files with hashes and primary URLs. The historical pipeline report links older commit `64a9483...`; our earlier data guide explains those reported measurements and the released experiment tables.

Recommendation for the team: first reproduce a selected real-data subset end-to-end at one pinned version, with a standard Marin model/optimizer recipe. Add cyber as a new versioned source and controlled mixture experiment. Do not simultaneously change dedup, tokenizer, optimizer, architecture and domain weights and call the outcome a replication.

The actual DAG and its data contracts

Entry point: `experiments/datakit/reference_pipeline.py`.



Several branches run before filtering; expensive attributes can be computed on text ultimately removed. Execution order and reusable artifacts matter for cost estimation. The final store joins decisions rather than expecting an earlier step to rewrite every text file.

1. Source registry: named components and dependency chains

`lib/marin/src/marin/datakit/sources.py` defines `DatakitSource(name, normalize_steps, rough_token_count_b)`. Its terminal `normalized` property points to the last step; earlier steps may share downloads across family components. The rough count is **Llama-3-based upstream accounting**, not retained inventory under an arbitrary new tokenizer.

`experiments/datakit/scripts/trigger_sources.py` selects explicit registry names, submits download/normalization chains, skips cached completed artifacts and has a `--list-pending` mode. Do not run it with no selection on a new cluster merely to inspect the registry: the default covers all sources.

A staged command template, in an already configured Marin/Iris environment, is:

```
# SELECTED_SOURCES contains exact comma-separated registry names.
python -m experiments.datakit.scripts.trigger_sources \
  --sources "$SELECTED_SOURCES" --list-pending
python -m experiments.datakit.scripts.trigger_sources \
  --sources "$SELECTED_SOURCES" --downloads-only
python -m experiments.datakit.scripts.trigger_sources \
  --sources "$SELECTED_SOURCES"
```

These commands were inspected, not run here. Credentials, object storage, source access, worker environment and selected source revisions remain prerequisites. A complete `StepSpec` graph can still point to inaccessible upstream or internal artifacts.

2. Normalization: stable identity is a scientific control

`lib/marin/src/marin/datakit/normalize.py` discovers supported raw formats, maps the chosen text column to `text`, preserves metadata unless explicitly dropped, and moves the original ID to `source_id`. `source_id` **here is the upstream record ID, not necessarily the registry source name.** Keep both concepts in our release manifest.

`generate_id(text)` hashes UTF-8 text with **xxh3_128**, yielding a zero-padded 32-character hex ID. It is not a URL hash or an arbitrary source-row identifier. Whitespace runs exceeding 128 characters are compacted under a declared rule, and IDs are recomputed if text changes. This is more precise than saying "normalize whitespace": it does not authorize arbitrary stripping that damages code indentation, tables or logs.

The default exact-normalization mode deduplicates records by content ID and sorts each partition by ID. A no-dedup mode exists for explicit purposes. Default target partition bytes are 256MiB. Important content settings enter `NORMALIZE_IDENTITY_ATTRS`; changing them should change artifact identity. Input-file capping through `FERRY_TEST_MAX_FILES` is a **testing hook**, not a legitimate full-release selection policy unless separately manifested.

Required pilot evidence: raw/normalized counts, encoding replacement/empty-record counters, compacted examples, source-ID preservation, duplicate side output and stable IDs on rerun. Metadata URLs can change without text changing; preserve that provenance even when choosing one content representative.

3. Global exact and fuzzy dedup are different operations

`experiments/datakit/global_exact_dedup.py` combines selected normalized sources and chooses canonical exact representatives. Sparse duplicate attribute shards contain removed IDs; a missing sparse file can legitimately mean no exact duplicates. Preserve the source manifest so a consumer distinguishes that convention from a missing required input.

The reference DAG's `MinhashConfig` uses **286 permutations, 26 bands, word 5-grams, seed 42 and a 500,000-character cap**. These generate candidates, not final semantic equivalence. The current fuzzy chain estimates large connected components, joins actual text, then verifies containment. The README describes a **75% distinct word-3-gram containment** rule, longer representatives first, plus bounded search for large groups. It includes character/group limits that can affect removal decisions.

Do not reconstruct this as "remove all LSH-connected documents except one": transitive candidate chains and short templated text can cause massive over-deletion. Preserve candidate, comparison and representative information for review. Cyber examples differing by a version, IP, command flag or patch line can be meaningfully distinct despite near-identical wording.

The inspected store exempts **16 named registry sources from fuzzy removal**, while exact dedup and decontamination still apply. This is a versioned exception policy, not an automatic exemption for every new structured/cyber source. Test errors before adding our own exemption; an exemption changes available unique data and the learned mixture.

4. Decontamination: more than a 13-gram overlap slogan

`lib/marin/src/marin/datakit/decon.py` and `decontamination_steps()` implement a shared eval Bloom filter **plus an eval-hash sidecar**, then source/global common-feature drop sets, document marking and reports. The reference constants include 13 whitespace-word n-grams, threshold 0.5, **minimum two distinct matched features** with special short-document handling, and a nominal Bloom false-positive target 10^{-9}.

Paragraphs are blank-line-delimited; n-grams span single newlines. Numeric-only and short-text handling are explicit in the implementation. Common-feature drop sets try to suppress ubiquitous boilerplate matches before computing overlap. A Bloom hit alone is not a certified semantic contamination verdict. Read the current feature policy and match/index logic

rather than transplanting only the threshold.

Supported stage targets are `decon-bloom`, `decon-drop`, `decon-mark` and `decon-report`. The `--mark-sources` flag lets a pilot mark a subset **without changing the full source set that produced common-feature filters**. Replacing `--sources all` with a tiny source set can change the drop-set policy and output identity.

Freeze eval corpus manifests and exclusions before preparing training data. The code documents a remaining region-derived eval-path identity issue: byte-identical data may still re-key across storage regions. Record both content hashes and execution paths; a green cache hit is not a substitute for content verification.

5. Topic embeddings: current Luxical versus historical Harrier

The current DAG imports `experiments/datakit/embeddings/luxical/pipeline.py`. It pins `DatologyAI/luxical-one` revision `474cfeb959dd473b3d1cd61da630f566037e69e2` and `luxical_one_rc4.npz`. It samples document text using a declared 32Ki-character input policy and stores **192-dimensional int8 vectors**. Retain its dequantization convention and model/input settings; changing them changes clusters.

The historical report instead describes Harrier 1,024-dimensional embeddings. A 40-topic label produced by Luxical is not necessarily the same semantic cell as historical Harrier `c14`. Old mixture weights cannot be applied by index to new centroids.

`cluster/domain/v0/train.py` trains spherical FAISS k-means with K=5000 and derives coarser views (40/1000) by agglomerative merging; assignment stores IDs and distances in co-partitioned shards. Seeds, thread configuration, sample and centroid bytes matter. The code notes that seeded numerical operations can vary across machine/thread configurations. For an exact corpus reconstruction, replicate centroid bytes rather than assuming a seed alone reproduces them.

6. Quality scoring has two paths—do not mix them

The inspected `reference_pipeline.py` invokes `score_normalized`, a token-only pooled transformer path. It consumes the corpus token IDs and verifies both tokenizer compatibility and the normalize-source identity. It uses begin/middle/end windows, averaging their scores so a repeated boilerplate prefix cannot dominate the whole document.

Crucially, `_predict_batch` applies calibration with no content-type input and **refuses per-type calibration** in this path. A separate **fusion-score → content-type →** `bucket.py` route supports type-conditioned calibration. The directory README discusses both generations; that is not proof the reference DAG executes both. Select the scorer/checkpoint/calibration that matches the exact caller.

The rubric is source-blind and content-type-aware: prose, code, math, multilingual, structured and other. It asks whether a document is a good example of its own type, rather than treating clear English prose as the definition of quality. The README records **5,578 oracle labels** for a described scorer generation; model/label artifacts live under object-store paths. Their public availability has not been established here.

`QualityPin` separately hashes model artifacts and calibration. A model directory contains `.eqx`, vocabulary-remap JSON, metadata JSON and calibration. Renaming a model/version tag without preserving compatible bytes is not reproduction. Bucketing uses edges **0.2/0.4/0.6/0.8**, yielding bands 0–4; the raw oracle labels 1–5 and these calibrated bands are different variables.

For cyber, keep generic intrinsic quality separate from cyber relevance and factual/verifiability attributes. A valid short advisory may be valuable but too small a pool to dominate training. See the cyber extension for a concrete design.

7. Store construction checks positional alignment

`experiments/datakit/store/datakit_store.py` aligns shard basenames, then compares **IDs in row order** across tokenization, decontamination, topic and quality attributes. The store aborts on mismatches; it does not safely "join whatever file has the same number of rows." Sparse exact/fuzzy duplicate attributes have their own conventions.

It removes contaminated/exact/fuzzy-marked records, applies declared exemptions, spills per-bucket token runs to local disk, then writes Levanter caches with aligned TensorStore writes. Each attempt has a unique directory and commits a completion sidecar last. Retries can ignore partial attempts and reuse complete ones.

The `ClusteredStoreData` artifact records tokenizer, cluster view, quality edges, bucket token/document counts, selected sources, exemptions and counters. These are the inputs for mixture feasibility. The production configuration can request large temporary machines (the inspected store worker has 96 CPUs, 700g RAM, 900g disk; 192 tasks). These are implementation resource settings, **not requirements that our smaller pilot must copy**. Profile and shrink them.

Marin's 128,256-row vocabulary requires token IDs wider than uint16: budget a uint32 token payload where that is the chosen format. The earlier 2-bytes/token example applies only to a compatible smaller vocabulary. Luxical's 192-dimensional int8 embedding payload is about 192GB per billion documents before IDs/metadata, whereas a 1,024-dimensional BF16 payload is about 2.048TB; the historical and current annotation storage budgets differ.

If we train with TorchTitan/Megatron, export a trainer-compatible store from the same retained documents and validate boundaries/token IDs. That port is a distinct step; a Levanter cache path is not a Megatron `.bin/.idx` dataset.

8. Mixture and epoching: inspect the actual schedule

`grug/moe_hero_ep/harrier_mix_2026_08_18.py` adopts an existing store URI and validates its tokenizer, weights and availability. In the inspected version it requires **three phase vectors: initial, main and cooldown**, although the earlier released experiment export also contains one-/two-phase histories. The initial/main split follows `108000/390251` of target total; pretrain totals 15T and cooldown 3.75T. The code checks **cumulative per-cell epochs ≤8**, not only each phase's epochs separately.

We also recomputed the schedule metadata: 200 cells, about 23.106T available tokens, phase budgets approximately 5.189T/9.811T/3.750T, and a maximum cumulative repetition of about 7.020. These are arithmetic checks of the published configuration, **not a fresh scan proving the underlying store is accessible or unchanged**.

Its model can use simulated epoching for cheaper proxies, with a stated analytic FLOP threshold. The simple principle is to scale down both availability and exposure, preserving D_b/U_b. Above the configured threshold it uses real pools. Context switches keep shuffle blocks and token budgets explicit so sequence-batch changes do not silently alter mixture histories.

Rebuilding a smaller/different corpus invalidates the old `available_tokens` and may violate epoch caps. Old weights are a candidate prior, not an instruction to sample nonexistent tokens. Recompute availability and validate a new mixture before training.

A concrete replication sequence for our team

Deliverable 1: dependency inventory. Resolve source revisions/access; exact tokenizer; selected historical/current pipeline; quality labels/model/calibration; embeddings and centroids; eval corpus manifests; storage and worker runtime; model/optimizer recipe. Mark missing bytes separately from available code. The replication contract lists these fields.

Deliverable 2: normalized pilot. Stratify about 1M documents across selected sources, lengths, quality/type labels and cyber sources. Save representative examples and duplicate/contamination fixtures. Do not call sample mode a complete reproduction: `SMOKE_SCALE` uses K=64 and coarser 8/16 views, whereas full mode uses K=5000 and a 40-topic view.

Deliverable 3: end-to-end pilot DAG. After staging an accessible quality checkpoint and a materialized normalized sample, a template is:

```
# Run inside the configured Marin/Iris worker environment; placeholders must be resolved.
python -m experiments.datakit.reference_pipeline \
  --mode sample --sample-prefix "$NORMALIZED_SAMPLE_PREFIX" \
  --sources all --quality-model "$QUALITY_MODEL_DIR" \
  --quality-model-version "$QUALITY_CONTENT_VERSION" \
  --pool-workers 16 --max-concurrent 2
```

This has not been executed here. The sample prefix must contain the expected artifact layout; a directory of arbitrary JSONL is insufficient. Omit pre-staged centroids only when intentionally training a new sample taxonomy. The source code shows an Iris launcher; replace Marin-specific cluster/storage names with our configured infrastructure rather than copying a private bucket.

Deliverable 4: integration corpus. Prepare roughly 10B retained tokens, a reconciled stage report, bucket capacities, repeatable loader state and several short model/eval runs. Inject a known duplicate across sources, a benchmark-overlap example and an attribute-order mismatch; verify detection/failure. These tests exercise actual integrity properties, not just whether a job exits zero.

Deliverable 5: scaled release. Measure per-stage documents/s, tokens/s, RAM, spill bytes and retry behavior. Extrapolate from that pilot, with skew/retention uncertainty, to the selected 1.6–1.9T retained-token capacity objective. Freeze source membership before final global dedup/store release. Adding a new cyber source changes cross-source exact/fuzzy decisions and must produce a new final release.

Deliverable 6: mixture learning and hero gate. Use exposure-matched proxies, fresh/branch-aware validation, target and general guardrails, then the optimizer transfer protocol. A functioning pipeline is necessary but does not establish that the new cyber mixture improves the model.

Reproduction acceptance

Accept a data release only when every source and transform is pinned; ID/source lineage is recoverable; all attribute joins validate; duplicate and contamination audits pass; centroids/scorer/calibration are compatible; retained-token counts reconcile; cumulative exposure respects caps; the trainer/evaluator sees the same intended tokenizer and masks; and restart reproduces sampling state. Record what is a historical replication, a current-code run, or our extension in the release metadata.

No full Datakit execution, annotation-model training or prepared corpus download has occurred in this repository. The source audit identifies the exact work required, including artifacts that code alone does not supply.